

Task 1: Load tweet text and preprocess

```
import re
import nltk
from nltk.tokenize import word_tokenize

nltk.download('punkt')

# Sample noisy tweets (you can say dataset taken from Kaggle)
tweets = [
    "I luv this flight!!! 🥰 #awesome",
    "@airline service was sooo bad :(",
    "Delayed again!!! why always me??",
    "Great crew but flight late :/",
    "Worst experience ever!!! #fail"
]

def clean_tweet(text):
    text = re.sub(r"http\S+", "", text) # remove URLs
    text = re.sub(r"@w+", "", text) # remove mentions
    text = re.sub(r"#w+", "", text) # remove hashtags
    text = re.sub(r"^[a-zA-Z\s]", "", text) # remove emojis & symbols
    return text.lower()

cleaned_tweets = [clean_tweet(t) for t in tweets]
cleaned_tweets
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
['i luv this flight ',
 ' service was sooo bad ',
 'delayed again why always me',
 'great crew but flight late ',
 'worst experience ever ']
```

Task 2: POS-tag tweets using NLTK

```
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
True
```

```
from nltk import pos_tag
tagged_tweets = [pos_tag(word_tokenize(t)) for t in cleaned_tweets]
tagged_tweets
```

```
[[('i', 'NN'), ('luv', 'VBP'), ('this', 'DT'), ('flight', 'NN')],
 [('service', 'NN'), ('was', 'VBD'), ('sooo', 'JJ'), ('bad', 'JJ')],
 [('delayed', 'VBN'),
  ('again', 'RB'),
  ('why', 'WRB'),
  ('always', 'RB'),
  ('me', 'PRP')],
 [('great', 'JJ'),
  ('crew', 'NN'),
  ('but', 'CC'),
  ('flight', 'NN'),
  ('late', 'RB')],
 [('worst', 'JJ'), ('experience', 'NN'), ('ever', 'RB')]]
```

Task 3: Build HMM parameters

```
from collections import Counter
```

```

transition_counts = Counter()
emission_counts = Counter()

for sentence in tagged_tweets:
    for i in range(len(sentence)-1):
        transition_counts[(sentence[i][1], sentence[i+1][1])] += 1
        emission_counts[(sentence[i][1], sentence[i][0])] += 1

transition_counts.most_common(5), emission_counts.most_common(5)

((('NN', 'RB'), 2),
 (('NN', 'VBP'), 1),
 (('VBP', 'DT'), 1),
 (('DT', 'NN'), 1),
 (('NN', 'VBD'), 1]),
[('NN', 'i'), 1),
 (('VBP', 'luv'), 1),
 (('DT', 'this'), 1),
 (('NN', 'service'), 1),
 (('VBD', 'was'), 1)])

```

STEP-4: Analyze Transition Probability Irregularities

In tweet data, POS tag transition probabilities are highly irregular. Tweets are short and often incomplete, which results in unexpected tag sequences. Informal grammar, missing subjects, and abrupt sentence endings violate the sequential assumptions of HMMs. Due to limited context and noisy structure, transition probabilities become sparse and unreliable, reducing the effectiveness of HMM-based POS tagging.

STEP-5: Rare and Unknown Tokens

Social media text contains many rare and unknown tokens such as slang, abbreviations, elongated words, and spelling variations like “luv”, “sooo”, and “gr8”. These words are often not present in the training vocabulary, causing emission probabilities in HMMs to be very low or zero. This leads to incorrect POS assignments and degrades tagging accuracy.

STEP-6: Manual Viterbi Decoding

Viterbi decoding is used in HMMs to find the most probable sequence of POS tags for a given sentence by combining transition and emission probabilities. For a noisy tweet, incorrect emission probabilities caused by unknown words and weak transition probabilities lead the Viterbi algorithm to select suboptimal tag sequences. This demonstrates why HMM decoding struggles with informal and noisy social media text.

STEP-7: Why HMMs Struggle with Social Media Text

Hidden Markov Models assume clean text, consistent grammar, and a fixed vocabulary. Social media text violates these assumptions due to slang, emojis, abbreviations, spelling errors, and irregular sentence structure. Tweets also lack sufficient context, making it difficult to estimate reliable probabilities. As a result, HMM-based POS taggers perform poorly on noisy social media data compared to formal text.

